

Received October 2, 2020, accepted October 16, 2020, date of publication October 26, 2020, date of current version November 5, 2020.

Digital Object Identifier 10.1109/ACCESS.2020.3033942

Bandwidth Control Mechanism and Extreme Gradient Boosting Algorithm for Protecting Software-Defined Networks Against DDoS Attacks

HASSAN A. ALAMRI^{1,2} AND VIJEY THAYANANTHAN¹

¹Department of Computer Science, Faculty of Computing and Information Technology, King Abdulaziz University, Jeddah 21589, Saudi Arabia

²Department of Computer Science, College of Computing in Al-Qunfudhah, Umm Al-Qura University, Makkah 21955, Saudi Arabia

Corresponding author: Hassan A. Alamri (halamri0135@stu.kau.edu.sa)

ABSTRACT Software-Defined Networking (SDN) is an emerging network architecture that addresses the limitation of the traditional network by providing centralized management through a central controller that decouples the control and data planes. However, this development has made the controller a severe target for malicious users to execute attacks such as Distributed Denial of Service (DDoS) attacks. Several schemes have been proposed to mitigate DDoS attacks in SDN, but the challenges still exist. This paper proposes a DDoS mitigation scheme for SDN to ensure accurate attack detection and efficient network resource utilization. The scheme employs two stages: a bandwidth control mechanism and Extreme Gradient Boosting (XGBoost) Algorithm. The bandwidth control mechanism utilizes an adaptive bandwidth profile-based threshold and bandwidth control algorithm that trigger the XGBoost algorithm in case of threshold violations. The use of multiple bandwidth profiles in setting the threshold ensures the threshold's adaptivity to consider the network traffic variation and reduce the packets drop ratio, which shows an outstanding result. The XGBoost algorithm classifies network traffic flow that violates a set threshold into normal or abnormal traffic. We evaluated the performance of our scheme using CICDDoS2019, NSL-KDD, and CAIDA datasets. Furthermore, we validated our proposed solution in real-time with the SDN environment. The results obtained show that our scheme protects SDN against DDoS attacks with high accuracy, low error, and efficient utilization of the network resources. The proposed system achieved 99.9% accuracy in detecting DDoS attacks with a low false-positive rate of 0.0002% in SDN.

INDEX TERMS Bandwidth, distributed denial-of-service (DDoS) attack, extreme gradient boosting (XGBoost) algorithm, security, software-defined networking (SDN).

I. INTRODUCTION

The tremendous increase in network applications and connected devices on the internet has resulted in high network complexity in terms of management, configuration, and scalability [1]. The Internet of Things, virtualization, cloud services, big data, and other evolving technologies demand changes in the conventional network architecture to handle the massive data and evolution in technology [2]. Software-defined networking (SDN) addresses the limitations of conventional networks by providing features such as

programmability, logically centralized control, and simplified management [3].

SDN is an emerging network architecture that decouples the network control plane from the data (forwarding) plane [4]. The control plane is centralized as a software control logic implemented in a controller, while the forwarding plane remains on the forwarding devices for forwarding traffic [5]. OpenFlow is a widely-used communication protocol between these planes through an application program interface called the southbound interface. SDN uses the logical centralization of the controller to preserve a network-wide view to perform the forwarding decision, policy enforcement, and management [6].

The associate editor coordinating the review of this manuscript and approving it for publication was Ghufraan Ahmed.

For the SDN workflow, switches in the data plane process incoming packets based on the flow rule matches in a flow table. If the rule does not exist in the table, a Packet_in message is sent to the controller containing header information. The controller then creates the rule and action to be sent to the switch through the Packet_out message. This workflow provides effective and flexible traffic management and control; however, the centralized control approach is a target of major security attacks [7], [8].

The centralized nature of the SDN architecture causes numerous network security problems [9]. The primary security concern in any centralized network architecture is the distributed denial-of-service (DDoS) attacks. DDoS attacks can cripple the whole network by saturating the bandwidth and other network resources [10]. In the SDN context, DDoS exploits the table-miss occurrence to overwhelm the controller with Packet_in messages. The adversary can utilize various tools to forge the packet fields and send massive table-miss traffic along with normal traffic to the SDN environment, resulting in the depletion of bandwidth and other resources. Without an efficient protection mechanism, the controller is flooded with forged flows to create new flow rules and actions, consequently leading to the failure of the controller [11], [12].

Over the past several years, extensive research has been conducted to mitigate DDoS attacks in SDN. The most recent solutions are based on two approaches, namely, thresholding and machine learning approaches. The first approach employs a static threshold based on various flow parameters, such as average packet and byte rate, as a mechanism to detect DDoS attacks in SDN. However, this approach leads to low detection efficiency and an increase in the packet drop ratio because it disregards the bursty nature of the current normal traffic. Moreover, the threshold's accuracy may result in a significant underutilization of resources (i.e., bandwidth). In other words, if the static threshold is set to less than the available bandwidth, then the bandwidth is underutilized. Similarly, if the threshold is set above the required consumption rate, attackers can exploit it to perform an attack.

In the machine learning approach, network traffic features are periodically collected in a sliding window manner. Then, a trained machine learning model performs traffic classification to detect an attack. However, continuous feature collection and traffic classification can cause other security concerns owing to the difficulty in accurately setting the time interval for the sliding window method. That is, collecting traffic features in short time intervals increases processing and communication overhead. However, a long-time interval may allow an adversary to launch an attack before the classification model detects the attack. Additionally, preparing the learning model requires an up-to-date dataset representing a realistic attack and normal traffic. Furthermore, to avoid misleading detection accuracy, adequate data preprocessing and proper training and testing stages are required.

We propose a DDoS attack detection and mitigation system based on a bandwidth control mechanism and the extreme

gradient boosting (XGBoost) algorithm to address the limitations of the aforementioned approaches. The contributions of this work are listed:

- i. We propose an adaptive bandwidth profile-based threshold to detect attacks and reduce the packet drop ratio accurately.
- ii. We employ a bandwidth control algorithm that works in conjunction with the adaptive threshold to penalize flows exceeding the threshold in order to prevent bandwidth depletion and maintain network function even during an attack.
- iii. We implement a trigger-based detection and classification approach that employs the XGBoost algorithm to evaluate and classify traffic into normal or abnormal traffic with high accuracy and low error.
- iv. We test the effectiveness of the preprocessing steps we perform on the datasets with other work used the same dataset and learning algorithms.
- v. We validate the performance of the proposed system in mitigating DDoS attacks in real-time with the SDN environment.

The contributions mentioned above can address the limitations of the thresholding and machine learning approaches. In particular, the drawbacks of high packet drop ratio and low detection efficiency of the thresholding approach are addressed by replacing a static threshold with an adaptive one. This proposed adaptive threshold is accurately set using bandwidth profiles that measure traffic variation experienced by networks over time. Consequently, normal bursty traffic is distinguished from DDoS high-volume attack traffic, which reduces the legitimate packets drop ratio and increases the detection efficiency. Moreover, the proposed bandwidth control mechanism resolves the bandwidth underutilization caused by an inaccurate setting of the static threshold.

In terms of the limitations of the machine learning approach, the processing overhead and delay resulting from continuous feature collection and traffic classification are resolved using the proposed trigger-based approach. This approach works together with the adaptive threshold and the bandwidth control algorithm to trigger the machine learning model only when the traffic is highly suspected to be attack traffic. Up-to-date and realistic datasets are used to train and test the machine learning models. Besides, adequate preprocessing steps are performed on the selected datasets to prevent misleading results and overfitting problems.

The remainder of this paper is organized as follows. Section II summarizes related works, and Section III outlines the proposed system design and architecture. Section IV illustrates the workflow of the proposed solution in detail, and Section V demonstrates the performance evaluation. Section VI presents the results of the proposed system evaluation. Section VII discusses and analyzes the results. Finally, Section VIII concludes the paper and presents ideas for future work.

II. RELATED WORK

Because DDoS attacks pose a serious threat to most SDN schemes, researchers have proposed a considerable number of DDoS detection and mitigation solutions. Most of these solutions can be classified as thresholding-based solutions [13]–[17] or machine-learning-based solutions [18]–[28].

A. THRESHOLDING APPROACH

Thresholding-based solutions [13]–[17] use machine learning algorithms or statistical methods to set a threshold value to detect attacks against SDN. In particular, when SDN traffic flow exceeds the predefined static threshold, it is considered as attack traffic.

A lightweight algorithm for protecting the SDN controller against DDoS attacks is proposed in [13]. This method involves setting a threshold based on the median of the flow parameters in a Transmission Control Protocol (TCP) flow. The parameters include the average packet rate, average byte rate, and flow duration placed in a data flow table. The controller creates the table to store internet protocol (IP) addresses with three counters: a packet counter, size counter, and duration counter. Once the flow is received, it is stored in the table, and the counters start. After that, the first threshold is set on the basis of the first counter, and if it is violated, another threshold is set based on the second counter. If this threshold is violated, the final threshold is set based on the third counter. If the final threshold is violated, a DDoS attack is detected. Otherwise, the timers are reset.

DDoS attack detection and mitigation scheme based on information-theoretic entropy and random forest (RF) is proposed in [14]. Simply, the proposed attack detection is based on setting a threshold that is calculated through traffic entropy. In other words, the entropy of the current window traffic is calculated to estimate the incoming traffic. If the average entropy of the feature exceeds the regular range, it is considered as suspected attack traffic. Then, the current window traffic is preprocessed and classified to distinguish between normal and abnormal traffic.

An adaptive DDoS attack detection method based on multiple kernel learning is proposed in [15]. A powerful feature extraction algorithm is used to extract five features from the traffic to detect attack traffic. Then, multiple kernel learning models are combined with the adaptive weights of each feature. The proposed algorithm effectively extracts the features based on the DDoS attack characteristics; however, the static threshold is set to each of the five features, yielding low accuracy and a high false-positive rate.

An effective threshold defense against the DOS attack on the SDN controller is proposed in [16]. Because the controller has a variable queue that queues all arriving packets, the authors set a threshold value to monitor the number of packets in this queue. In this method, each packet's source and destination IP addresses are stored in a hash table. A counter is then set in this table to count the number of packets arriving with the same IPs. If this count exceeds the threshold value,

a DoS attack is suspected. Thus, Time to Live (TTL) flag is set to 0 for that packet and all other packets identical to it. Then, the source is blocked as a mitigation process. Consequently, the controller processes only packets with TTL not equal to 0 and drops packets with TTL equal to 0. This algorithm works based on a timer; if the times go off, all counts are cleared and set to zero.

An efficient DDoS flood attack detection method using dynamic thresholding on flow-based network traffic is proposed in [17]. The proposed solution first extracts various traffic features to calculate four attributes based on DDoS attack characteristics. Then, to detect an attack, all four attributes must exceed the threshold value. The threshold is set by calculating the mean and variance of each attribute.

Despite the positive results achieved using thresholding-based DDoS mitigation solutions [13]–[17], these solutions use a static threshold as the detection mechanism based on various flow parameters. As previously mentioned, this method can produce low detection accuracy and underutilization of resources (i.e., bandwidth).

B. MACHINE LEARNING APPROACH

Machine learning algorithms are used in many security solutions [18]–[28] owing to their significant advantages and powerful features, such as high accuracy, continuous improvement, and the ability to handle a large amount of data. Moreover, single or hybrid machine learning algorithms are employed to detect DDoS attacks in SDN by classifying traffic into normal or attack traffic.

A DDoS detection approach based on multi-classifier ensemble learning is proposed in [18]. A heuristic algorithm is proposed based on singular value decomposition (SVD) to construct the detection system. In the experiment, the results of three machine learning algorithms using SVD and un-SVD, namely, k-nearest neighbor, bagging, and RF, are compared.

SGuard, a lightweight SDN safeguard architecture for DoS attacks, is proposed in [19]. SGuard is designed as a security application that works on top of the SDN controller. The proposed solution comprises three modules. The first module is an access control module, in which the true source of the packets is identified and traced to mitigate an attack. The second module is based on the self-organizing map (SOM) algorithm and performs classification to distinguish malicious traffic from normal traffic. The classification module is optimized in [16] by implementing feature ranking and selection algorithms. The third module is a data plane cache that resides between the SGuard and data plane to cache table-miss packets during a saturation attack temporarily.

A DDoS attack detection method for SDN based on the support vector machine (SVM) is proposed in [20]. In this method, six-tuple characteristic values are extracted and utilized for learning and classifying traffic into normal and abnormal traffic. The extracted features include speed of the source IP, source port's rate, standard deviation of the flow packets, deviation of flow bytes, speed of flow entries, and

pair-flow ratio. The results demonstrate a high accuracy rate and a low false alarm rate. However, the Internet Control Message Protocol (ICMP) traffic detection accuracy rate is low because ICMP packets do not contain the port address. Consequently, the value of the extracted features is zero.

OverWatch, a cross-plane DDoS attack defense framework with collaborative intelligence for SDN, is proposed in [21] to achieve fast and accurate DDoS attack detection and mitigation. The control and data planes jointly monitor DDoS attacks and react against them in SDN. A coarse-grained flow monitoring algorithm is employed in the data plane, and a fine-grained machine-learning-based attack classification algorithm is used in the control plane. Sensors in the data plane continuously monitor the flow for any suspected flow. If any attack flow is detected, this module notifies the control plane by providing information about the flow. Then, the control plane leverages this information and activates the defense system, which is based on a machine learning algorithm.

Detection of DDoS attacks in software-defined networks is proposed in [22]. Snort was used in this work as the first level of security to detect signature-based attacks. Moreover, SVM and Deep Neural Network (DNN) learning models were used to detect anomaly-based attacks as the second security level. The SVM and DNN models were trained based on KDD Cup dataset. The IDS calculates the number of TCP pair-connections between the source and destination nodes. Afterward, this count is used as an input to the trained machine learning models to classify the new connections, whether normal or anomaly. The paper concluded that the DNN outperformed SVM with higher accuracy, recall, and precision.

Four machine learning algorithms, namely, the multilayer perceptron, SVM, decision tree (DT), and RF, are analyzed and compared in [23] to detect DDoS attacks in SDN. These algorithms are proposed to detect three types of DDoS attacks: flow table, bandwidth, and controller attacks. Results reveal that the DT generally outperforms all other algorithms because it requires the lowest processing time. RF achieves the highest accuracy; however, it requires a long processing time. In terms of the features, the port source is the most significant feature in the classification of anomalies, followed by the duration time.

Advanced SVM (ASVM)-based detection for DDoS attacks in SDN is proposed in [24]. ASVM is a multiclass classification technique that includes three types of traffic. This approach is an enhancement of the existing SVM algorithm for DDoS detection. ASVM addresses the problem of linear classification in SVM caused by the short distance between two hyperplanes and data falling into the margin. ASVM introduces slack variables that measure the distance between each data point and the margin of the plane, and classification error that maximizes the margin. In [20], the detection accuracy of ASVM reaches 97% and only a short time is required for the training and testing phases.

DDoS attack defense method in the SDN-based cloud is proposed in [25]. First, hybrid machine-learning-based on SOM and SVM is introduced to improve the flow classification. Second, an enhanced history-based IP filtering scheme (eHIPF) is developed to increase attack detection speed and rate. Finally, the abovementioned hybrid machine-learning-based defense and eHIPF were combined to create a novel DDoS attack defense for SDN-based cloud networks. The results of the study indicated that the proposed scheme is a practical approach.

SGS, a safeguard scheme for protecting the control plane against DDoS attacks in SDN, is proposed in [26]. SGS uses a controller's clustering method to deploy multiple controllers in the control plane. SGS comprises two main modules: (i) anomaly traffic detection deployed in switches in the data plane, and (ii) controller dynamic defense, which is implemented in the control plane. The first module distinguishes suspicious flows from legitimate flows using an improved back-propagation neural network method by adopting four-tuple feature vectors, such as variation in the asymmetric flow. The second module minimizes the effects of a DDoS attack by remapping the controller and providing the affected switches with an access control message to block the attack source.

DDoS attacks detection using machine learning is proposed in [27]. In this work, four supervised machine learning algorithms were evaluated: DT, SVM, Logistic Regression (LR), and KNN. All learning models were trained using a dataset that is collected from a student portal website. After performing the dataset preprocessing and feature selection, the machine learning algorithms were evaluated based on two performance metrics, which are Jaccard score and F1-score. The results showed that the SVM model attained superior scores in classifying DDoS attacks.

A flexible SDN-based architecture for identifying and mitigating low-rate DDoS attacks using machine learning is proposed in [28]. The IDS implemented in the proposed architecture was trained with a set of machine learning algorithms, namely J48, Random Tree (RT), REP Tree, RF, MLP, and SVM. The performance of these algorithms was evaluated using CIC DoS 2019 dataset. The results showed that the Multi-Layer Perceptron algorithm outperformed other models and gained the highest values in all performance metrics: accuracy, precision, recall, F1-score, and false alarm rate.

Although the above DDoS mitigation schemes [18]–[28] demonstrate high detection accuracy, the continuous traffic classification, and the statistic collection based on the sliding window method lead to processing and communication overhead. In other words, the collection of traffic features in small-time intervals increases processing and communication overhead. However, a long-time interval may allow an adversary to launch an attack before the traffic is classified, particularly for large networks. Furthermore, training a machine learning model requires extensive preprocessing and proper feature selection on the dataset.

III. PROPOSED SYSTEM DESIGN AND ARCHITECTURE

The proposed system is a DDoS attack mitigation scheme for SDN comprising bandwidth control mechanism and learning model based on the XGBoost algorithm. The bandwidth control mechanism consists of an adaptive threshold technique and a bandwidth control algorithm. The solution is designed to be implemented in the SDN control plane and comprises five modules, as illustrated in figure 1. These modules are the threshold setting module, bandwidth monitoring module, bandwidth control module, detection and mitigation module, and statistics module. Each module's role is explained in the subsequent subsections by describing the techniques and algorithms used in building the proposed solution.

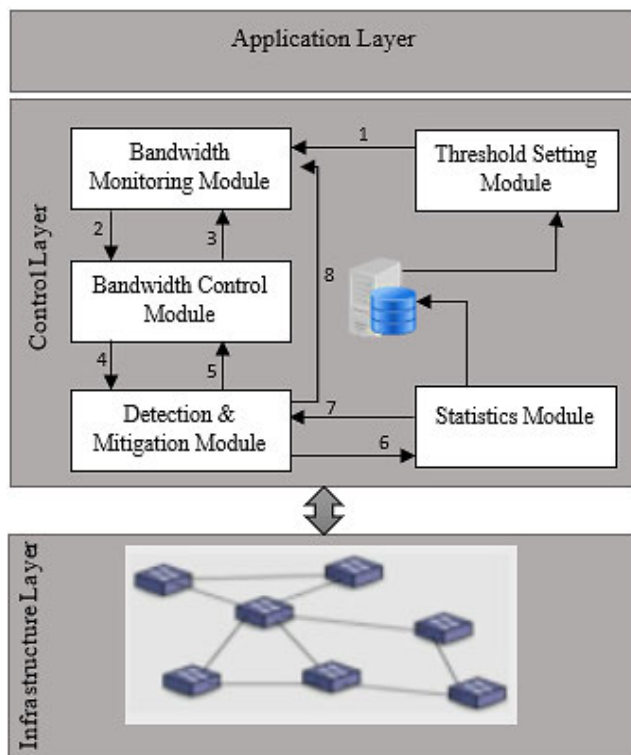


FIGURE 1. Design of the proposed DDoS detection and mitigation system.

A. ADAPTIVE THRESHOLD TECHNIQUE

Network traffic in any organization varies over time in compliance with the policies and business requirements of the organization. For example, the network traffic rate during the business-working hours is noticeably higher than that after the working hours of the organization. Therefore, to consider and detect high variations in network traffic, the proposed adaptive threshold technique is implemented based on the bandwidth profiles. Thus, the bandwidth threshold adaptivity is achieved through the switching between these three profiles over time. Without these profiles, the threshold becomes more likely as a static threshold that usually results in a high packet drop ratio of normal traffic. Such traffic variation may be caused by DDoS attacks or even normal bursty traffic.

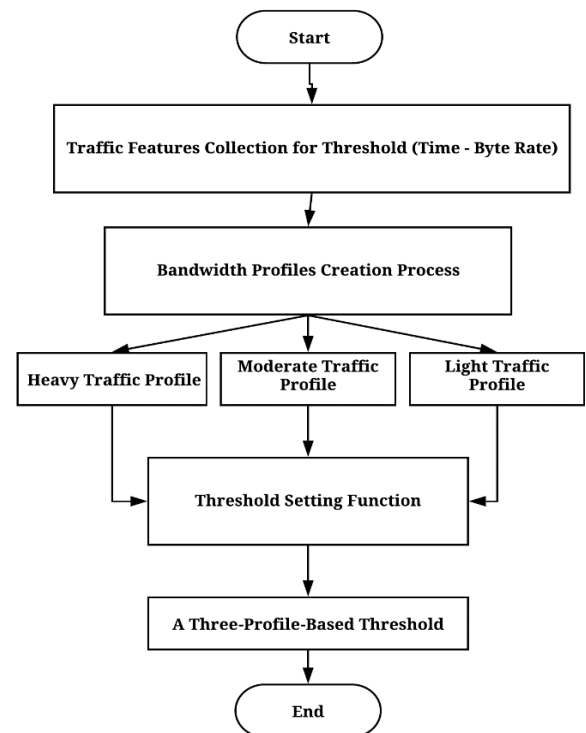


FIGURE 2. Adaptive threshold setting process for the proposed system.

The effectiveness of this proposed bandwidth multi profile-based threshold is evaluated and discussed in sections V and VI. Figure 2 shows the process of setting the proposed adaptive threshold, which is discussed in detail in the following subsections.

1) TRAFFIC FEATURE COLLECTION FOR THRESHOLD

The features used by the proposed threshold technique are the time and byte rate, which reflect the variation in network traffic over time. Therefore, the first step in the threshold setting process is importing the traffic features from devices in the network infrastructure layer, as illustrated in figure 2. Thus, the statistics module in the proposed design periodically fetches the features mentioned above and stores them in a comma-separated value (CSV) file named *Threshold_Features* in the data store. Once the data required for the threshold become available in the data store, the next step of the process (bandwidth profile creation) begins.

2) BANDWIDTH PROFILE CREATION

Based on the time and byte rate features, the proposed threshold setting module classifies and divides the data traffic into three profiles: heavy traffic, moderate traffic, and light traffic profiles, as depicted in figure 2. The heavy traffic profile represents the time and byte rate at which the network usually experiences a heavy load at a specific time. An example of a heavy load is traffic during business working hours, where all services of an organization are running and utilized by employees and clients. In contrast, the light traffic profile

represents the time and byte rate at which the network usually experiences a low traffic level. An example of light traffic is network traffic after midnight, which is supposed to be lower than other profiles, as most of an organization are likely inactive. The moderate traffic profile represents a moderate level of network traffic at a specific time. After the bandwidth profiles are categorized and created, the threshold setting process's next step is the threshold setting function, as discussed below.

3) THRESHOLD SETTING FUNCTION

The threshold setting module in the proposed design passes the bandwidth profiles created in the previous step, bandwidth profile creation, to the threshold setting function, as illustrated in figure 2. Then, the threshold setting function calculates the threshold based on these profiles using (4), yielding a three bandwidth-profile-based threshold. As a result, threshold adaptivity is achieved through the switching between these three bandwidth profiles over time. The threshold is adaptively increased and decreased according to the level of network traffic that the network experiences at a specific time.

Let μ be the mean, σ^2 be the variance, β be the threshold parameter, x_i be the data samples (byte rate).

$$\text{Threshold} = (\mu + \sigma^2) * \beta \quad (1)$$

$$\mu = \frac{1}{n} \sum (x_i) \quad (2)$$

$$\sigma^2 = \frac{1}{n} \sum (x_i + \mu)^2 \quad (3)$$

$$\text{Threshold} = \left(\frac{1}{n} \sum (x_i) + \frac{1}{n} \sum (x_i + \mu)^2 \right) * \beta \quad (4)$$

The threshold is periodically updated using the steps mentioned above, shown in figure 2, to achieve accurate adaptivity for the network variation over time.

B. BANDWIDTH CONTROL ALGORITHM

The proposed bandwidth control algorithm works in conjunction with the defined threshold to minimize an attack's effects, such as bandwidth depletion, even when the network is under attack. This algorithm uses a penalty mechanism that limits the bandwidth by half (1/2) for flow exceeding the threshold. Then, the bandwidth is linearly increased for the penalized flow, as expressed in (5). Meanwhile, the detection and mitigation module in the proposed design begins classifying the traffic to detect an attack. The detection and mitigation module is activated only when the maximum threshold violation counter (TVC) is exceeded. The TVC counts the number of violations of the bandwidth threshold for a specific flow. The algorithm below is proposed for this purpose.

Let F be the traffic flow, $BW(t)$ be the bandwidth during a time interval (t) , $thre$ be the threshold, a be the bandwidth reduction parameter, and b be the bandwidth linear increment

parameter.

$$w(F_{i+1}) = \begin{cases} BW(t) * a & \text{if } F \geq thre \text{ where } (a = 1/2) \\ BW(t) + b & \text{if } F < thre \text{ where } (b > 0) \end{cases} \quad (5)$$

C. TRAFFIC CLASSIFICATION MODEL

The final stage in developing the proposed solution involves building a traffic classification model. Inspired by the methods proposed in [25], [29], [30], an ensemble machine learning algorithm is employed on the traffic flows to classify the traffic into normal traffic or DDoS attack traffic. As reported in [29], a detection scheme based on hybrid or ensemble machine learning algorithms outperforms the scheme based on a single machine learning algorithm, achieving higher accuracy. Accordingly, the XGBoost algorithm is employed in the proposed detection and mitigation module because of its high classification accuracy, low false-positive rate, high speed, and algorithmic simplicity.

1) EXTREME GRADIENT BOOSTING ALGORITHM

The XGBoost algorithm was proposed by Chen and Guestrin [31]. It is a supervised machine learning algorithm based on the ensemble method and was developed as an enhanced form of the gradient boosting algorithm [32]. The XGBoost algorithm employs additive techniques to develop a powerful learning model by combining weak learners' predictions. In addition to its speed and high performance, the XGBoost classifier overcomes the overfitting problem and attains full utilization of computational resources. These advantages result from simplifying the objective functions that permit a combination of regularization and predictive terms, as well as capacity for parallel execution in the training phase [31]. In terms of the XGBoost algorithm's procedures, the first learner is fitted to the entire data. Then, the second learner is fitted with the first learner's errors. This process is continuously performed until a stopping condition is met, at which point the final prediction model is created by adding the predictions of all learners [31]. Equation (6) represents the prediction method at a specific step:

$$f_i^{(t)} = \sum_{n=1}^t f_n(x_i) = f_i^{(t-1)} + f_t(x_i) \quad (6)$$

where $f_t(x_i)$ is the prediction at step t , $f_i^{(t)}$ and $f_i^{(t-1)}$ are the predictions at step t and $(t-1)$, respectively, and x_i is the input value.

To avoid the overfitting problem without affecting the computational speed of the model, (7) evaluates the goodness of the model:

$$Obj^{(t)} = \sum_{k=1}^n l(\bar{y}, y) + \sum_{k=1}^t \alpha(f_i), \quad (7)$$

where n is the number of observations, l is the loss function, and α is the regularization function, which is expressed in (8) as follows:

$$\alpha(f) = \delta T + \frac{1}{2} \sigma \mu^2, \quad (8)$$

where δ is the minimum loss necessary to partition the leaf node further, σ is the regularization parameter, and μ is the vector of scores in the leaves.

2) TRAFFIC CLASSIFICATION BASED ON XGBoost

The XGBoost model is appropriately trained using realistic and up-to-date datasets that contain the most recent and various types of DDoS attacks and normal traffic, as discussed in section V. Thereafter, the model is implemented in the proposed system. The detection and mitigation module in the proposed design is activated only when a specific flow exceeds the maximum TVC. The maximum TVC is the maximum number of threshold violations for a specific flow before the XGBoost model is activated (we assume that $\text{Max_TVC} = 3$). When the module is activated, it calls the statistics module to collect 20 features of the flow that exceeds the Max_TVC . These collected features are then saved in the `Traffic_Classification_Features.CSV` file in the data store. After that, the XGBoost model uses this data file and performs traffic classification as trained. The attack traffic is traced and mitigated based on the classification, while the bandwidth control module controls normal bursty traffic. This approach prevents the bandwidth from being overwhelmed by that flow and ensures the fairness of other flows. The experiment, datasets, features, and performance evaluation are described in section V.

IV. WORKFLOW OF PROPOSED SOLUTION

After presenting the algorithms and techniques used in the proposed system, this section discusses the workflow and integration modules of the proposed design. Figure 3 presents the proposed system's workflow, while algorithm 1 displays the steps performed to detect and mitigate DDoS attacks in SDN.

As illustrated in figure 3, there are three phases in the workflow of the proposed system. These phases are the monitoring phase, bandwidth control phase, and detection and mitigation phase.

Monitoring phase: The bandwidth monitoring module monitors the predefined threshold to detect any violation. Upon threshold violation detection, the TVC is incremented by 1, and the bandwidth control module is triggered to activate the next phase.

Bandwidth control phase: Once the bandwidth control module is triggered, it penalizes the flow that exceeded the threshold by limiting the bandwidth by half ($1/2$) and then linearly increasing it to prevent bandwidth depletion. The bandwidth control module then triggers the detection and mitigation module if Max_TVC is exceeded; otherwise, the operation is returned to the monitoring phase.

Detection and mitigation phase: Upon triggering the detection and mitigation module, the trained XGBoost model performs traffic classification to detect the attack. When the traffic flow is classified as an attack, this module traces and blocks the attack flow by sending a REST message to the controller to block the attack flow and clean the flow table.

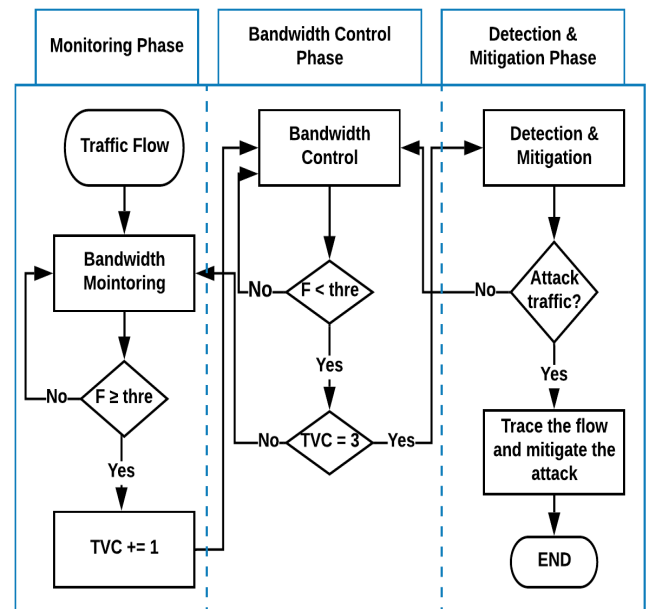


FIGURE 3. Workflow of the proposed DDoS detection and mitigation system.

Otherwise, the traffic is considered normal, and the operation is returned to the bandwidth control phase and the TVC is reset.

V. PERFORMANCE EVALUATION

In this section, we present a performance evaluation of the proposed solution in detecting and mitigating DDoS attacks in the SDN context. The following subsection A presents the classification algorithms evaluation in order to determine the best algorithm that outperforms others to be employed in our solution. Datasets description, data preprocessing stage, performance metrics, and the evaluation approach are discussed in this subsection. In addition, subsection B presents the evaluation of the proposed system in real-time with the SDN environment. Thus, the classification model that outperforms other algorithms in subsection A as well as the proposed bandwidth control mechanism is validated in real-time in the SDN environment.

A. CLASSIFICATION ALGORITHMS EVALUATION

This subsection presents the performance evaluation of the classification algorithms, which are Logistic Regression (LR), Naive Bayes (NB), Decision Tree (DT), Random Forest (RF), and XGBoost. the datasets description, preprocessing stage, performance metrics, and evaluation approach are elaborated in subsections 1–4 below.

1) DATASETS DESCRIPTION

An important stage of building an attack detection and mitigation scheme is performance evaluation because it determines the efficiency and robustness of the proposed scheme. Thus, a dataset with identical characteristics to real normal

Algorithm 1**Input:** Receiving Flow Traffic**Output:** Free Attack SDN Environment

(1) Set

Thre $\leftarrow$ Bandwidth ThresholdP $\leftarrow$ Period $T_i \leftarrow$ Time Interval $BW_{(i+1)} \leftarrow$ Bandwidth at T_i $F_i \leftarrow$ FlowTVC $\leftarrow$ Thre Violation CounterMax_TVC $\leftarrow$ (Max_TVC=3) $F_{Stat} \leftarrow$ Flow statisticsThre_{Stat} $\leftarrow$ Threshold Flow statistics $a \leftarrow$ Bandwidth Reduction Parameter $\mu \leftarrow$ mean $\sigma^2 \leftarrow$ VarianceM $\leftarrow$ Mitigation Policy for (F_i)D $\leftarrow$ Detection & Mitigation Module

(2) Calculate Thre

Input: Time & Byte rate in P**Output:** Three profile-based threGet Thre_{Stat}

Calculate traffic profiles

For each profile:

Thre = $(\mu + \sigma^2) * \beta$

(3) Start Bandwidth Monitoring Module

while $F_i > \text{Thre}$ **do** $BW_{(i+1)} = F_i * a$

TVC = TVC + 1

IF TVC $\geq$ Max_TVC **then**

Start D

Input: F_{Stat} **Output:** Normal or AttackGet F_{Stat} **IF** $F_i \leftarrow$ Attack **then**

Apply M

Else

GoTo Step (3)

End IF**Else**

GoTo Step (3)

End IF**End While****End**

traffic and ddos attack traffic flows is required. Consequently, we evaluated the performance of the classification algorithms using three datasets, namely CICDDoS2019 [33], NSL-KDD [34], and CAIDA-DDoS attack 2007 [35].

a: CICDDoS2019 DATASET

This dataset is considered the most recent and realistic DDoS attack dataset developed at the University of New Brunswick,

Canadian Institute for Cybersecurity, Canada. It covers normal traffic and the most common and up-to-date types of DDoS attack traffic. To resemble real traffic and network characteristics, the dataset was developed using a testbed comprising a set of equipment and software, such as PCs, servers, firewalls, routers, switches, and traffic-generating tools.

The background network traffic was generated using the B-Profile approach discussed in [35]. For attack traffic, 11 DDoS attack profiles were executed using a third party's tools and packages. The DDoS attack types in this dataset are depicted in figure 4 [32]. Additionally, 80 features were extracted using the CICFlowMeter extractor [36], [37].

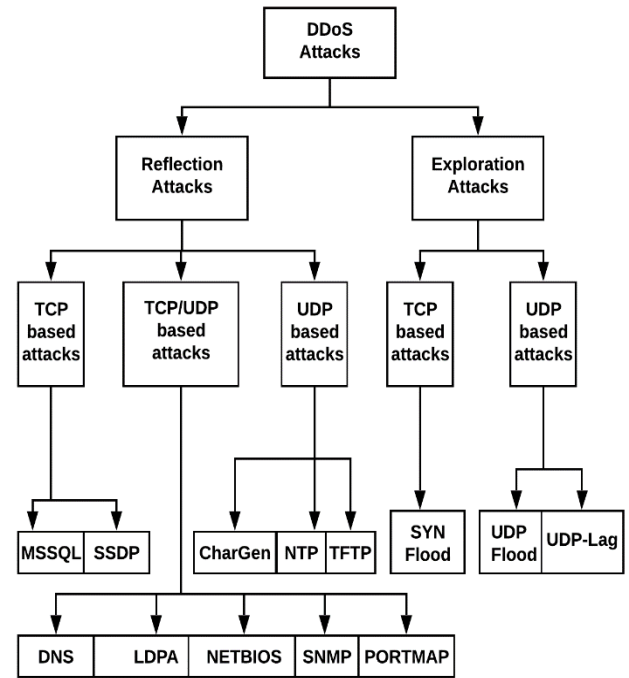


FIGURE 4. Distributed denial-of-service (DDoS) attack taxonomy.

b: NSL-KDD DATASET

NSL-KDD dataset [34] is a refined version of the KDDCup'99 dataset to address some inherent problems discussed in [39]. This dataset's main improvement over KDDCup'99 is removing the redundant records, which solves the bias problem during the traffic classification. The NSL-KDD dataset covers four different attack categories, which are Denial of Service (DoS), User to Root (U2R), Remote to Local (R2L), and Probing attacks [40]. The category considered in this paper is the DoS records since the scope of this paper is to propose a DDoS mitigation scheme in SDN. The DDoS attacks types covered in this dataset are Back, Neptune, Smurf, Teardrop, Land, Pod, Apache2, Mailbomb, and Processtable attacks. Further, the NSL-KDD dataset contains 41 features for each record representing the normal and attack traffic characteristics.

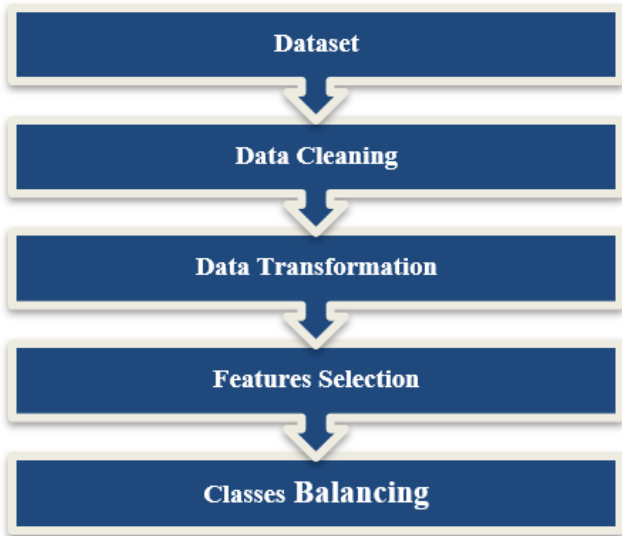


FIGURE 5. Data preprocessing steps prior to performance evaluation.

c: CAIDA-DDoS ATTACK 2007 DATASET

CAIDA dataset contains roughly an hour of anonymized traffic traces from a DDoS attack on august 4, 2007. This type of DDoS attack aims to overwhelm computing resources and the bandwidth in order to block access to the server connected to the internet. The one-hour trace is split into several 5-minute pcap files. CAIDA dataset includes only DDoS traffic to the targeted victim and the responses while the background network traffic has been removed. Therefore, the normal traffic used in this dataset is extracted from the CICDDoS2019 dataset for classification purposes. This dataset's DDoS attack type is mainly the ICMP flooding attack and a little TCP-syn attack traffic. The traces are anonymized using Cryptopan prefix-preserving anonymization using a single key [35]. Table 1 displays a summary of the three datasets used in this paper and the number of extracted samples of each class.

2) DATASETS PREPROCESSING

Dataset preprocessing is a crucial step prior to the performance evaluation of any classification algorithm. It is a critical stage, particularly for machine learning-based mitigation schemes, to improve classification accuracy and prevent misleading results. Figure 5 presents the preprocessing steps that were applied to the datasets, which include data cleaning, data transformation, feature selection, and balancing the class distribution.

In this study, the CICDDoS2019, NSL-KDD, and CAIDA DDoS attack 2007 datasets were subjected to the abovementioned steps to validate its suitability for evaluation. First, the data cleaning phase removed any errors and missing or null values as well as irrelevant values, such as the string data type. Second, data transformation, such as normalization, standardization, and discretization, was applied to some

TABLE 1. Summary of the datasets (CICDDoS2019, NSL-KDD, AND CAIDA).

Dataset	Creation Date	Binary Classes	Multiclass Classes	DDoS Attack /Normal	No. of Extracted Samples
CICDDoS2019	2019	✓	✓	Normal	48890
				DNS	48415
				LDAP	48891
				MSSQL	48930
				NetBIOS	46776
				NTP	47290
				SSDP	48891
				Syn	48890
				UDP	48890
				TFTP	46666
NSL-KDD	Refined version of KDD,99 that was created in 1999	✓	✓	UDP-Lag	48891
				Normal	45138
				Neptune	45495
				Smurf	45649
				Teardrop	45081
				Land	45017
				Pod	45097
				Apache2	45317
CAIDA	2007	✓	N/A	Mailbomb	45845
				Processtable	45792
				Normal	58621
				ICMP	60023

attributes to meet some of the evaluated machine learning algorithms' requirements.

The next step is feature selection, which is an important process having a significant impact on the proposed algorithm's performance and accuracy. Information Gain (IG) was used to perform feature selection on the datasets. The information gain evaluates the worth of an attribute by measuring the information gain with respect to the class using (9) [41]. Then, the features with a rank of 1 and above are selected because they demonstrate outstanding performance and high accuracy in traffic classification. The total number of features selected from the CICDDoS2019 dataset is 20 out of 80 features, as illustrated in table 2. In NSL-KDD dataset, 24 out of 41 features are selected for this paper's evaluation, as shown in Table 3. As for the CAIDA dataset, the features selected are source IP address, destination IP address, source port number, destination port number, protocol, timestamp, and length.

Let IG be the information gain, C be the class, F be the feature, and E be the entropy. The information gain is calculated using the following expression:

$$IG(C, F) = E(C) - E(C|F) \quad (9)$$

The final phase of dataset preprocessing is balancing the distribution of the dataset classes. An imbalanced dataset refers to a dataset with an unequal distribution of classes in the training dataset. Therefore, the model only performs well in terms of predicting the dominant classes, which leads to misleading accuracy. To overcome this problem, a dataset class-resampling technique is needed. The most popular dataset class resampling techniques include over-sampling, under-sampling, and hybrid methods [42]. The over-sampling method mainly involves increasing the size of an imbalanced dataset by duplicating the minority instances

TABLE 2. Selected feature description (CICDDoS2019 dataset).

S/No	Features	Description	Rank
1	Protocol	Protocol type	2.56
2	Average Packet Size	Average size of a packet	2.396
3	Min Packet Length	Minimum length of a packet	2.3959
4	Fwd Packet Length Mean	Mean size of a packet in the forward direction	2.3925
5	Avg Fwd Segment Size	Average size observed in the forward direction	2.3925
6	Fwd Packet Length Min	Minimum size of a packet in the forward direction	2.3918
7	Packet Length Mean	Mean length of a packet	2.3829
8	Fwd Packet Length Max	Maximum size of a packet in the forward direction	2.3041
9	ACK Flag Count	Number of packets with ACK	2.197
10	Flow Bytes/s	Number of bytes per second	2.1561
11	Source Port	Source port number	2.0377
12	Max Packet Length	Maximum length of a packet	1.929
13	FWD Packets/s	Number of forwarded packets per second	1.5146
14	Flow Packets/s	Number of flow packets per second	1.5128
15	Subflow Fwd Bytes	Average number of bytes in a subflow in the forward direction	1.3233
16	Total Length of Fwd Packets	Total size of a packet in the forward direction	1.3233
17	Flow Duration	Duration of the flow in microseconds	1.1717
18	Fwd IAT Total	Total time between two packets sent in the forward direction	1.1641
19	Source IP	Source IP address	1.0627
20	Destination IP	Destination IP address	1.0608

of some classes for equal distribution among the classes. The synthetic minority over-sampling technique (SMOTE) is the most widely used type of over-sampling method [43].

In contrast to over-sampling methods, the under-sampling methods reduce the size of the dataset by removing minority instances of some classes to equalize the instances of the number of all classes. Tomek links (T-link) are a popular under-sampling method that balances the dataset classes [44]. Hybrid methods use a combination of under-sampling and over-sampling methods to eliminate the overfitting problem caused by over-sampling as well as a lack of training data resulting from the under-sampling. The SMOTETomek method [45], which we used in this study to balance the datasets, is a combination of SMOTE and Tomek links. The SMOTE is initially used to increase the minority instances of some classes, and then the T-link is used thereafter.

3) PERFORMANCE METRICS

The detection and mitigation scheme, which is based on the classification algorithms, was evaluated using a set of performance metrics to determine the models' effectiveness and robustness in classifying attack and normal traffic. Our evaluation's performance metrics included the accuracy, precision, recall, F1-Measure, and false-positive rate.

The confusion matrix is a table depicting the performance of a classification model by presetting and relating the predicted and actual observations on the classes. It has four central values that are used to calculate the abovementioned

TABLE 3. Selected feature description ((NSL-KDD dataset).

S/N	Features	Description	Rank
1	Src_bytes	No. of bytes from source to destination	2.14
2	Service	Destination service used	1.91
3	Flag	connection Status	1.88
4	Diff_srv_rate	Rate of connections to different services	1.87
5	Same_srv_rate	Rate of connections to the same service	1.84
6	Dst_bytes	No. of bytes from destination to source	1.76
7	Count	No. of connections to the same destination.	1.73
8	Dst_host_diff_srv_rate	Rate of connections to different services	1.72
9	Dst_host_same_srv_rate	Rate of connections to the same services	1.71
10	Dst_host_srv_count	No. of connections with the same port number	1.66
11	Dst_host_srv_error_rate	Rate of connections with same flag among the connections aggregated in dst_host_srv_count	1.62
12	Dst_host_error_rate	Rate of connections with same flag among the connections aggregated in dst_host_count	1.62
13	Error_rate	Rate of connections with same flag among the connections aggregated in the count	1.57
14	Srv_error_rate	Rate of connections with same flag among the connections aggregated in srv_count	1.57
15	Logged_in	Login Status	1.48
16	Dst_host_same_src_port_rate	Rate of connections to the same source port	1.39
17	Srv_count	No. of connections to the same service	1.35
18	Dst_host_count	No. of connections with the same destination host	1.28
19	Dst_host_srv_diff_host_rate	Rate of connections to different destination hosts	1.27
20	Protocol_type	Protocol type	1.26
21	Dst_host_error_rate	Rate of connections with same flag REJ, among the connections aggregated in dst_host_count	1.21
22	Dst_host_srv_error_rate	Rate of connections with flag REJ, among the connections aggregated in dst_host_srv_count	1.20
23	Srv_diff_host_rate	Rate of connections to different destination machines among the connections aggregated in srv_count	1.18
24	Duration	Duration of the connection	1.12

performance metrics. These values are listed in table 4 and defined as follows:

- True Positive (TP): Total actual positive observations that are correctly predicted as positive.
- False Negative (FN): Total actual positive observations that are incorrectly predicted as negative.
- False-positive (FP): Total actual negative observations that are incorrectly predicted as positive.
- True Negative (TN): Total actual negative observations that are correctly predicted as negative.

Based on the defined confusion matrix values, the aforementioned performance metrics were defined and calculated.

Accuracy is the ratio of the total number of positive observations to the total number of all observations. It can be

TABLE 4. Confusion matrix.

Confusion Matrix		Predicted Class	
		Positive	Negative
Actual Class	Positive	(TP)	(FN)
	Negative	(FP)	(TN)

calculated using (10) as follows:

$$Accuracy = \frac{(TN + TP)}{(TP + FP + TN + FN)} \quad (10)$$

Precision, also called the positive predictive value, is the ratio of the number of correct observations of positive samples to the total number of all positive observations. It can be calculated using (11) as follows:

$$Precision = \frac{TP}{(TP + FP)} \quad (11)$$

Recall, also called sensitivity, or TP rate, is the ratio of the number of positive observations correctly classified to the total number of actual observations. It can be calculated using (12) as follows:

$$Recall = \frac{TP}{(FN + TP)} \quad (12)$$

F1-Measure is the weighted average of recall and precision. It can be calculated with (13) as follows:

$$F1 - Measure = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (13)$$

False-Positive Rate (FPR), also called specificity, is the ratio of normal observations to the total number of normal observations misclassified as attacks. It can be calculated using (14) as follows:

$$FPR = \frac{FP}{(FP + TN)} \quad (14)$$

4) EVALUATION APPROACH

The proposed classification algorithms are evaluated based on two aspects: preprocessing stage and the traffic classification. Each evaluation aspect is elaborated in detail. The Python-based Scikit-learn machine learning libraries [46] executed on a Windows 10 operating system with 12 GB RAM and Intel (R) Core (TM) i7-6500U CPU at 2.50 GHz were used for the evaluation experiment. As previously mentioned, data preprocessing is a crucial stage of building a robust and accurate machine learning-based detection model because it improves the accuracy of the model in classifying traffic. To evaluate the effectiveness of our preprocessing phases described earlier in this section, we compared the results obtained in this paper with the results obtained in [33], which was the only study that used the same dataset for DDoS traffic classification to date. The present study is the first to evaluate the preprocessing steps performed on the abovementioned dataset to the best of our knowledge. In [33], only

feature selection preprocessing step was performed, where the authors used the random forest regressor. Features were selected per class based on the weight and mean of each feature. In other words, the importance of each feature in the entire dataset was calculated, and the final result was achieved by multiplying the average standardized mean value of each feature split on each class with the corresponding importance value of the feature. Thereafter, the authors of [33] evaluated the performance of four machine learning algorithms NB, LR, DT, and RF to measure the precision, recall, and F1 score metrics. In the present study, we evaluated the performance of the same algorithms to validate the effectiveness of our preprocessing steps, including feature selection. The results of this evaluation are presented in section VI.

To determine performance of the XGBoost algorithm in this study, each dataset was duplicated in two copies. The class label in the first copy was changed to binary classes (normal and attack classes) to perform binary classification. The class label in the second copy of the CICDDoS2019 dataset was multiclass, including Normal, DNS, LDAP, MSSQL, NetBIOS, NTP, SSDP, UDP, SYN, TFTP, and UDP-lag, to perform multiclass classification. In NSL-KDD dataset, the label of the class in the second copy dataset was multiclass, including Back, Neptune, Smurf, Teardrop, Land, Pod, Apache2, Mailbomb, and Processtable, to perform multiclass classification. As for CAIDA DDoS attack 2007 dataset, we only perform the binary classification since the dataset mainly has one type of attack, which is the ICMP attack [47], [48], in addition to the normal traffic that is included from the CICDDoS2019 dataset.

Thereafter, the XGBoost classifier was evaluated against the NB [49], LR [50], DT [51], and RF [52] classifiers.. The performance was evaluated using a 10-fold cross validation, where each dataset was randomly split into ten parts. One part was used for the testing set, and the remaining parts were used as the training set. To reduce the bias, variance, and overfitting problem, this procedure was repeated for each part of the divided dataset until the final part was reached.

All algorithms were evaluated on the basis of classification report metrics to measure the performance of each classifier in classifying different samples using binary classification, multiclass classification, and FPR. The classification report metrics included accuracy, precision, recall, and F1-Measure. The results of this evaluation are presented in section VI.

B. PROPOSED SYSTEM EVALUATION IN SDN ENVIRONMENT

This subsection presents the performance evaluation of the classification model (XGBoost) and the bandwidth control mechanism proposed in this paper in real-time with the SDN environment. The experimental setup and evaluation approach are outlined in the subsequent subsections.

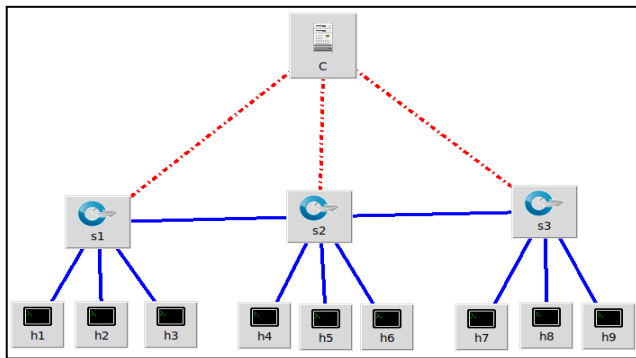
1) EXPERIMENTAL SETUP

The experiment was conducted using VMware Workstation 15 with two processors, 4 MG RAM, and a hard disc

TABLE 5. Parameters setting used in the experiment.

Parameter	Type/Value
Virtual Machine	VMware Workstation 15
Operating System	Ubuntu 20.04 LTS
Number of Processors	2
RAM	4MG
Hard dick	25 GB
Network Emulator	Mininet
Controller	Ryu
Switch Type	open vSwitches
Number of Switches	Three
Number of hosts	Nine
IP address range for hosts	10.0.0.1 – 10.0.0.9
Normal traffic generating tool	iPerf3
attack traffic generating tool	Hping3
Packet type	UDP
Destination port id	5001
Source port id	45209 & 51468
Experiment Duration	30 (s)
Total normal packet sent	5350
Total Attack packet sent	192586
Topology type	Tree

of 25 GB. Moreover, the Mininet emulator was used to create the SDN network topology on Ubuntu 20.04 LTS. The Ryu controller was used to manage and control the network. This controller is an open-source component-based SDN framework implemented in Python [53]. Mininet is a network emulator that creates a realistic virtual network, switches, hosts, and applications running on a single Linux machine [54]. The SDN environment in this work comprised one Ryu controller (c), three open vSwitches (s1–s3), and nine end-hosts (h1–h9), as shown in figure 6. The Hping3 and iPerf3 tools were used to generate attack and normal network traffic, respectively. The parameter setting used in the experiment is presented in Table 5.

**FIGURE 6.** Topology of the SDN environment used in the experiment.

2) EVALUATION APPROACH

This section discusses the approach conducted to evaluate the classification model (XGBoost) and the bandwidth control mechanism in real-time with the SDN environment depicted in figure 6.

a: CLASSIFICATION MODEL (XGBoost)

To evaluate the trained model of CICDDoS2019 in the SDN environment, we employed the model and the

bandwidth control mechanism in the SDN simulated environment. In RYU controller, the trained classification model (XGBoost) is configured and started analyzing the traffic. In Mininet emulator, the normal traffic is generated from two hosts (h1-h2) to the target host (h9) using iPerf3 tool. Then, we generated the DDoS attack traffic from host (h4) to the same target host (h9) using Hping3 tool with attack rate that can go up to 500 Mb/s. The results of the classification and detection accuracy is presented in section VI.

b: BANDWIDTH CONTROL MECHANISM

To validate the performance of the proposed bandwidth control mechanism in SDN real-time traffic, we have used two scenarios. In the first scenario, we employed the proposed bandwidth control algorithm in the SDN environment. Then, we generated normal traffic from hosts (h1 and h2) to the host (h9) using iPerf3 tool. Then we generated the attack traffic from host (h4) to the victim (h9) after five seconds with rate up to 500 Mb/s using Hping3 tool. In the second scenario, we performed the same testing approach used in the first scenario without employing the proposed control algorithm. Afterward, we extracted the bandwidth records from the SDN environment during the normal and attacked traffic for both scenarios. These records are then stored in external files to be parsed and plotted by measuring the bandwidth consumption with and without the proposed bandwidth control algorithm. The results and analysis of this experiment are presented in section VI.

c: BANDWIDTH PROFILE-BASED THRESHOLD

To evaluate the performance of the bandwidth profile-based threshold, we have used two scenarios. In the first scenario, we employed a single profile-based threshold, while in the second scenario, we employed a multi profile-based threshold (moderate and heavy profiles for simplicity). For both scenarios, Iperf3 tool was used to generate normal traffic from h1 to h5 with a different number of parallel sessions. For moderate traffic simulation, 20 parallel sessions were set in h1 to transmit traffic to h5 with different rates. For the heavy traffic, the number of sessions was increased to 50. Afterward, the moderate traffic was first generated, then the heavy traffic was generated after 15 seconds to test the threshold's adaptivity and accuracy in both scenarios. Finally, the packets drop ratio of applying the single profile-based threshold was compared to the multi profile-based threshold. The result of this evaluation is shown in section VI.

VI. RESULTS

This section presents the results obtained on the basis of performance evaluation conducted in this study. This evaluation was conducted to determine the performance of the bandwidth control mechanism and XGBoost-based classification. The results demonstrate the effectiveness of the proposed techniques in mitigating the DDoS attacks in SDN.

TABLE 6. Binary classification performance of classifiers. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).

Algorithm	Accuracy (%)			Precision (%)			Recall (%)			F1 score (%)		
	CICDDoS 2019	CAIDA	NSL-KDD	CICDDoS 2019	CAIDA	NSL-KDD	CICDDoS 2019	CAIDA	NSL-KDD	CICDDoS 2019	CAIDA	NSL-KDD
LR	80	93	93.51	85	91	94	80	90	94	80	91	94
NB	77	91.5	70	84	89	82	77	89	76	76	90	76
DT (ID3)	98.5	99	99.36	99	99.1	99.45	99	99.2	99.45	99	99.2	99.45
RF	98.55	99.1	99.52	99	99.3	99.66	99	99.3	99.66	99	99.3	99.68
XGBoost	99.7	99.9	99.92	100	100	100	100	100	100	100	100	100

A. PERFORMANCE OF CLASSIFICATION ALGORITHMS

This section presents and analyzes the performance evaluation results of the preprocessing steps performed on the datasets. Further, we present the binary and multiclass classification evaluation results of the XGBoost classifier, among other classifiers being evaluated on the datasets.

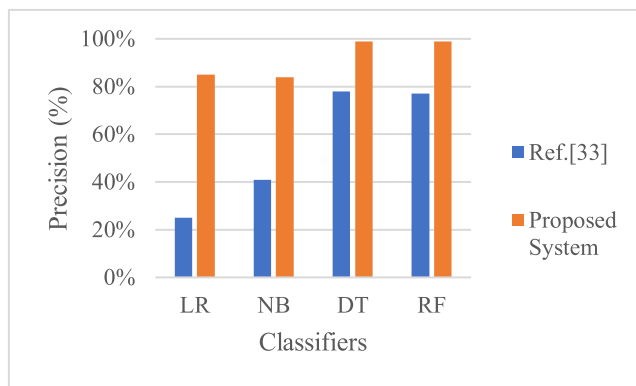
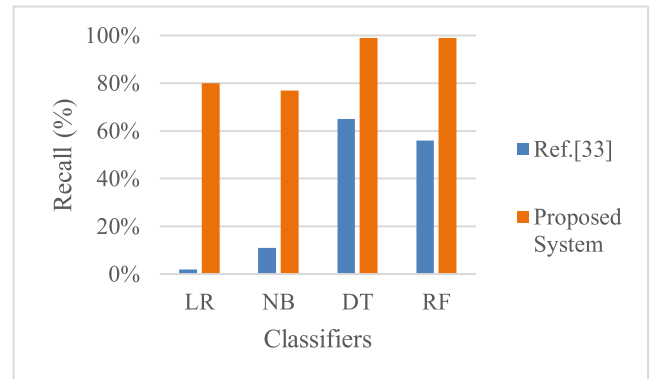
**FIGURE 7.** Preprocessing evaluation comparison between proposed system and system in [33] in terms of precision. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).

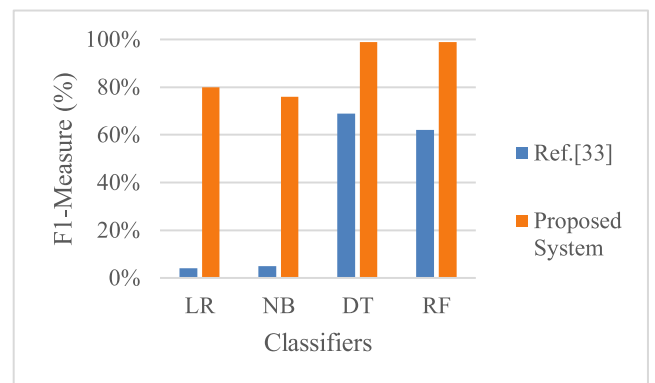
Figure 7 displays the classification precision of the detection models proposed in this paper and in [33]. The figure compared the influence of the preprocessing steps used in the present study and those used in [33] in terms of precision. The models proposed in this paper are superior to that reported in [33] owing to the performed preprocessing steps. The LR and NB algorithms presented in this study doubled the classification precision values of those in [33]. Moreover, DT and RF achieved 99% precision in this study, whereas they achieved 78% and 77% precision, respectively, in [33].

Figure 8 presents the recall values of the classifiers achieved in this study compared with those reported in [33]. LR and NB in [33] produced recall values of less than 12%, and in the proposed model, the recall values were almost 80%. Moreover, the recall values of the DT and RF classifiers in [33] were 65% and 56%, respectively, whereas they reached 99% in this study. The recall metrics improvement in this study was attributed to the preprocessing steps performed on the dataset.

Figure 9 shows the F1-Measure values of the classifiers achieved in this study compared with those reported in [33].

**FIGURE 8.** Preprocessing evaluation comparison between proposed system and system in [33] in terms of recall. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).

The F1-Measure of LR and NB only reached 5% in [33], whereas it reached 80% in the present study. Overall, the F1-Measure in this study was almost twice that obtained in [33].

**FIGURE 9.** Preprocessing evaluation comparison between proposed system and system in [33] in terms of F1-Measure. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).

In summary, figures 7–9 demonstrate the importance of the data preprocessing stage for classification algorithms' performance. The results imply that a model trained using effectively preprocessed data can obtain high performance in classifying and detecting different attack types. Thus, the preprocessing phases used in this study significantly enhanced

TABLE 7. Multiclass classification performance of classifiers. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).

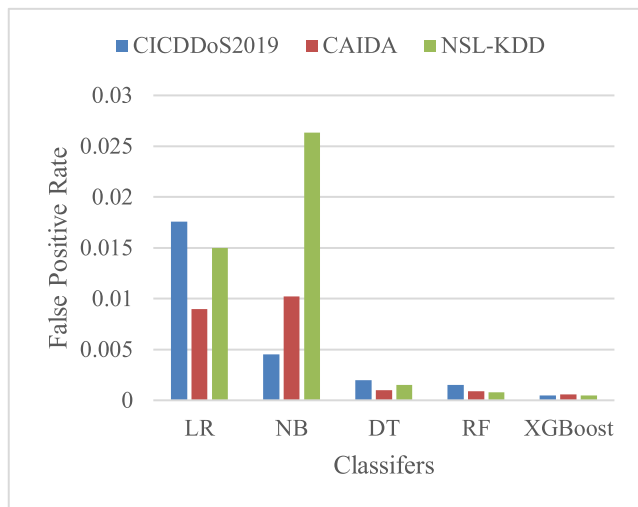
Algorithm	Average Accuracy (%)		Average Precision (%)		Average Recall (%)		Average F1 score (%)	
	CICDDoS2019	NSL-KDD	CICDDoS2019	NSL-KDD	CICDDoS2019	NSL-KDD	CICDDoS2019	NSL-KDD
LR	35	52	38	56	40	58	31	32
NB	31	40	22	48	20	48	15	33
DT (ID3)	85	99.69	89	94	89	95	89	95
RF	83	93	88	88	88	89	88	88
XGBoost	91.26	99.91	93	99	93	96	92	97

TABLE 8. Precision (%) of multiclass classification of all traffic instances with cicddos2019. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).

Algorithm	Normal	DNS	LDAP	MSSQL	NetBIOS	NTP	SSDP	Syn	UDP	TFTP	UDP-lag
LR	99	99	31	1	19	43	1	1	86	44	1
NB	1	6	16	1	84	1	1	42	61	25	3
DT	99	96	81	70	97	99	80	86	97	96	88
RF	99	98	80	75	97	99	80	73	98	97	68
XGBoost	100	100	86	85	96	100	81	89	99	99	90

TABLE 9. Recall (%) of multiclass classification of all traffic instances with cicddos2019. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).

Algorithm	Normal	DNS	LDAP	MSSQL	NetBIOS	NTP	SSDP	Syn	UDP	TFTP	UDP-lag
LR	6	68	99	1	45	93	1	1	41	99	1
NB	1	1	98	1	21	1	1	48	45	2	7
DT	99	97	96	61	98	99	74	85	96	97	87
RF	99	95	93	64	99	99	82	73	96	98	68
XGBoost	100	99	96	61	100	100	97	92	99	99	89

**FIGURE 10. The false-positive rate of each classifier in binary classification. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).**

the performance of the four evaluated algorithms compared to the results obtained in [33].

Figure 10 presents the False-Positive Rate (FPR) of each classifier being evaluated in this study. The figure indicates

that the XGBoost model can classify the traffic samples with the lowest FPR among all algorithms evaluated with CICDDoS2019, NSL-KDD, and CAIDA datasets with values of 0.0005%, 0.0003%, and 0.0006%, respectively. In contrast, LR and NB models obtained the highest FPR.

Tables 6-8 present the binary and multiclass classification evaluation results of the XGBoost classifier compared with LR, NB, DT, and RF classifiers. Table 6 shows the binary classification performance of the LR, NB, DT, RF, and XGBoost classifiers being evaluated using CICDDoS2019, NSL-KDD, and DAIDA datasets. The NB classifier produced the lowest performance with a range of 76%–84% with the CICDDoS2019 dataset, a range of 70-82% with NSL-KDD, and a range of 89-91.5% with the CAIDA dataset. On the other hand, the XGBoost classifier achieved the highest performance among all different classifiers in terms of accuracy, precision, recall, and F1-Measure with values of 99.7%–100% with CICDDoS2019 dataset, 99.92%-100% with NSL-KDD dataset, and 99.9%-100% with CAIDA dataset. As for DT and RF classifiers, they attained almost the same performance with all datasets with range of 98.5%-99.45% and 98.55%-99.68%, respectively.

Table 7 presents the multiclass classification performance of the classifiers evaluated in this study. The multiclass classification evaluation was conducted using the CICDDoS2019

TABLE 10. F1-measure (%) of multiclass classification of all traffic instances with CICDDoS2019. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).

Algorithm	Normal	DNS	LDAP	MSSQL	NetBIOS	NTP	SSDP	Syn	UDP	TFTP	UDP-lag
LR	12	81	48	1	27	61	1	1	55	61	1
NB	1	1	27	1	33	1	1	45	52	3	4
DT	99	96	88	65	97	99	77	86	97	97	87
RF	99	97	86	69	98	99	81	73	97	97	68
XGBoost	100	98	91	71	99	100	86	91	99	99	90

TABLE 11. Precision (%) of multiclass classification of all traffic instances with nsL-kdd. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).

Algorithm	Normal	Apache2	Back	Land	Mailbomb	Neptune	Pod	Processtable	Smurf	Teardrop
LR	38	37	33	29	93	94	36	41	76	82
NB	23	60	22	59	85	71	18	54	36	50
DT	99	98	50	99	100	98	95	99	100	100
RF	95	96	47	90	93	92	90	91	92	93
XGBoost	100	100	100	100	100	100	91	100	100	98

TABLE 12. Recall (%) of multiclass classification of all traffic instances with NSL-KDD. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).

Algorithm	Normal	Apache2	Back	Land	Mailbomb	Neptune	Pod	Processtable	Smurf	Teardrop
LR	39	40	36	25	95	96	42	50	73	80
NB	28	61	29	56.5	86	72	24	48	30	45
DT	99.2	98	72	99	98	98	93	95	100	94
RF	96	98	51	91	94	90	92	92.5	92	93
XGBoost	99.7	99	80	100	99.5	99	95	96.5	100	96

and NSL-KDD datasets since the CAIDA dataset mainly has one class of traffic, which is the ICMP flood. The XGBoost classifier achieved the highest average accuracy with CICDDoS2019 and NSL-KDD datasets with values of 91.26% and 99.91%, respectively. The DT and RF classifiers attained an average accuracy of 85% and 83%, respectively, with CICDDoS2019 dataset, while they achieved 99.69% and 93%, respectively, with NSL-KDD. Furthermore, XGBoost achieved the highest average precision, recall, and F1 score among all evaluated algorithms with a range of 92-99%, while DT and RF attained a range of 88-95% for the same metrics. Contrarily, the NB classifier yielded the lowest performance for all evaluated metrics with a range of 15-48%, followed by the LR classifier that obtained a range of 31-58%.

Tables 8-9 present the precision, recall, and F1-Measure of all classifiers in multiclass classification for each specific class using CICDDoS2019 dataset. Each class represents normal traffic and individual types of DDoS attacks.

Table 8 presents the precision obtained by the classifiers in multiclass classification for different attacks and normal instances. The XGBoost algorithm attained the highest precision level with a rate of 81-100%. In contrast, the NB classifier produced the lowest precision values in classifying normal traffic and attack traffic samples among all evaluated algorithms with rate of 1-84%. The DT

model recorded a precision of 70-99%, followed by the RF with 68-99%.

Table 9 presents the recall values of all algorithms for the multiclass classification of all normal and malicious classes. XGBoost achieved the highest recall with a range of 89-100% except for the MSSQL, where it gained 61%, followed by RF and DT with a range of 61-99%. The LR classifier achieved considerably better recall than NB in most instances, ranging from 1-99% and 1-48%, respectively, except for LDAP with NB, where it recorded 98%.

Table 10 presents the F1-Measure of the multiclass classification of normal traffic and DDoS attack instances. The XGBoost achieved the highest F1-Measure with a rate of 90-100% in all instances except for the MSSQL and SSDP, for which the algorithm achieved 71% and 86%, respectively. In contrast, the NB classifier obtained the lowest F1-Measure of 1%-52%, followed by LR, which recorded a 1-81% range. DT and RF attained an F1-measure of 65-99% and 68-99%, respectively.

Tables 11-13 present the precision, recall, and F1-Measure of all classifiers in multiclass classification for each specific class using NSL-KDD dataset. Each class represents normal traffic and individual types of DDoS attacks.

Table 11 presents the precision obtained by the classifiers in multiclass classification. The XGBoost algorithm attained

TABLE 13. F1-Measure (%) of multiclass classification of all traffic instances with NSL-KDD. (LR: logistic regression, NB: naive Bayes, DT: decision tree, RF: random forest).

Algorithm	Normal	Apache2	Back	Land	Mailbomb	Neptune	Pod	Processtable	Smurf	Teardrop
LR	10	10	8	4	90	93	8	20	69	5
NB	26	20	7	30	33	86	16	27	63	20
DT	99	99	67	99	99	99	90	99	98	96
RF	94	93	68	92	93	91.5	87	89	86	92
XGBoost	100	100	85	100	100	99.5	92	99.3	99	98

the highest precision level of 91–100%. In contrast, the NB classifier showed the lowest precision values in classifying normal traffic and attack traffic samples among all evaluated algorithms with 18-85%. The DT model recorded a precision of 50-100%, followed by the RF with 47-96%.

Table 12 presents each algorithm's recall values for the multiclass classification of all normal and malicious classes. XGBoost achieved the highest recall with a range of 95-100% except for the back class, where it gained 80%, followed by RF and DT with a range of 72-100% and 51-96%, respectively. The NB and LR classifiers attained the lowest recall values with a range of 28-86% and 25-96%, respectively.

Table 13 presents the F1-Measure of the multiclass classification of normal traffic and DDoS attack instances. The highest F1-Measure was achieved by XGBoost with a rate of 85–100% for all instances. In contrast, the NB classifier obtained the lowest F1-Measure with a rate of 7–86%, followed by LR, which recorded a range of 4-96%. The DT and RF classifiers attained an F1-measure of 67-99% and 68-94%, respectively.

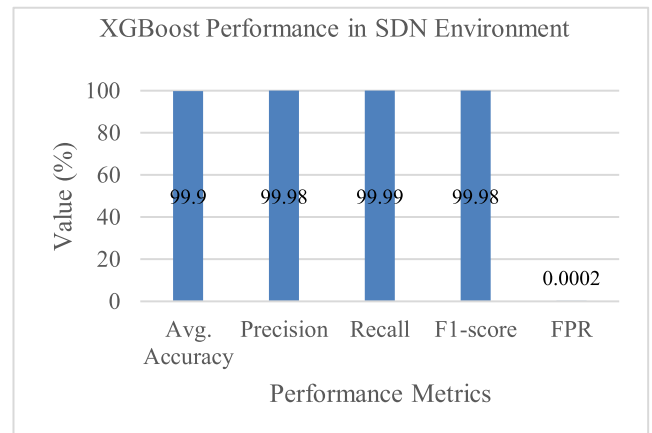
In summary, the results show that the preprocessing steps used in this work remarkably provided a better result than the steps used in [33]. The feature selection using information gain yields best-correlated features that enhance traffic classification results for the learning algorithms. As for the binary and multiclass classification, the performance of LR, NB, DT, RF, and XGBoost is attributed to their operational procedure and the ability to select the best parameters as well as the correlation of the features used from the dataset. Thus, NB and LR algorithms have the lowest values in all performance metrics compared to other algorithms. The assumption of features independency in NB leads to low performance while the selected features are dependent on the dataset. Detailed information about the operational procedures of NB is discussed in [55], [56]. LR is a linear algorithm that results in poor performance in multiclass classification, and it is not flexible to adapt to complex data. More details about LR operational procedures are discussed in [57].

The RF and DT models gained almost the same results since they perform the classification based on building trees. The main difference between TD and RF is the way of building the tree. DT builds a single tree with branches and leaves. Then, the decision is made through the leaves that represent the classes. RF is an ensemble of a set of trees, and the classification is performed by the majority voting of the results from each tree. More details about

DT and RF algorithms' operational procedures are covered in [51] and [52], respectively. XGBoost attained a superior performance compared to other algorithms due to the additive learning technique, which keeps improving weak learners' prediction. Also, XGBoost overcomes the overfitting problem and attains full computational resources utilization, as discussed in detail in section III.

B. PERFORMANCE OF PROPOSED SYSTEM IN SDN ENVIRONMENT

After evaluating the performance of XGBoost classifier using CICDDoS2019, NSL-KDD, and CAIDA datasets, this section presents the performance of the trained model (XGBoost) as well as the bandwidth control mechanism in real-time SDN traffic. Thus, the proposed system can be realized in real-time with the SDN environment.

**FIGURE 11.** XGBoost performance in real-time with SDN environment.

1) CLASSIFICATION MODEL (XGBoost)

In this section, we present the evaluation result of the XGBoost model in detecting the DDoS attack traffic with SDN real-time traffic. Figure 11 shows that the XGBoost model obtained an outstanding performance with an average accuracy of 99.9 %. Moreover, the model attained high precision and an F1-score of 99.98 % and recall value of 99.99 %. Furthermore, XGBoost recorded a very low false-positive rate of 0.0002 %. Consequently, employing XGBoost in the SDN environment protects SDN against DDoS attacks with high accuracy and low errors.

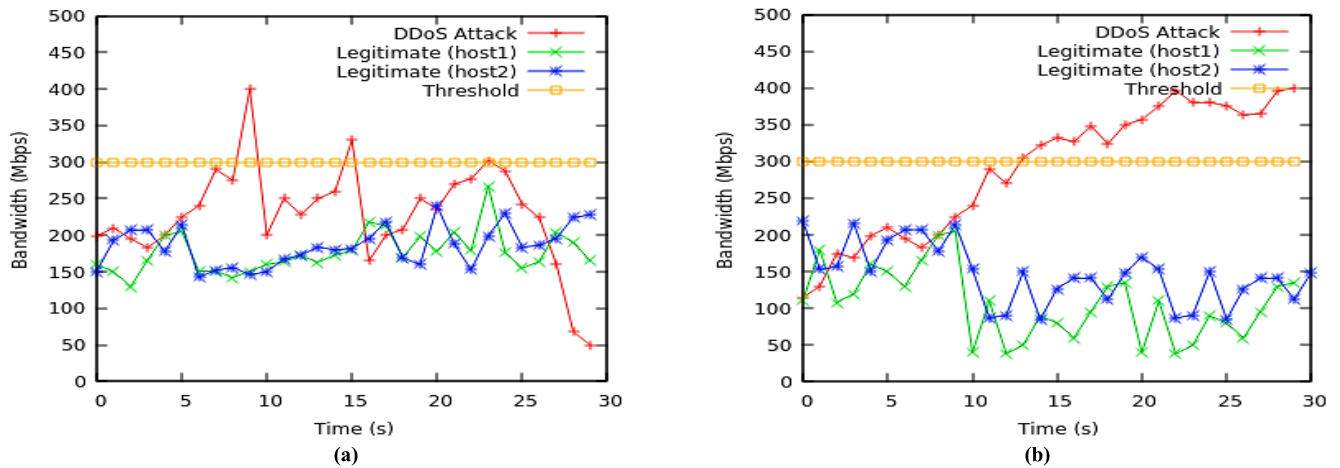


FIGURE 12. Bandwidth consumption. (a) Control algorithm enabled (Scenario 1); (b) Control algorithm disabled (Scenario 2).

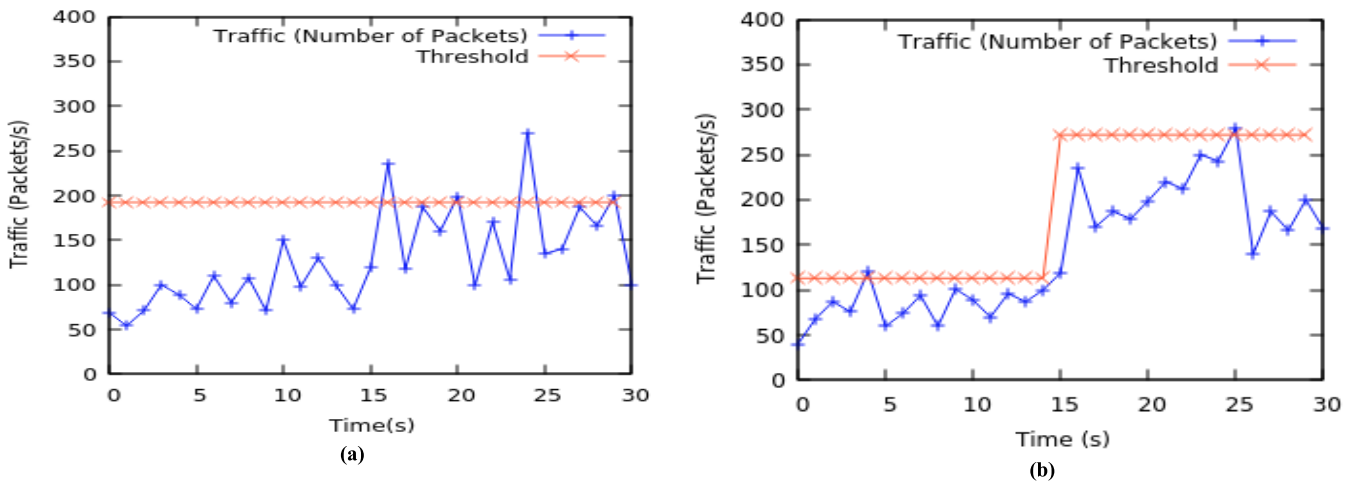


FIGURE 13. Bandwidth profile-based threshold. (a) Single profile-based threshold (Scenario 1); (b) Multi profile-based threshold (Scenario 2).

2) BANDWIDTH CONTROL MECHANISM

Figure 12 reveals the effectiveness of the proposed algorithm in controlling bandwidth consumption. Two scenarios are conducted in a real-time SDN environment, as discussed in section V. Figure 12 (a) shows that when the bandwidth control algorithm is applied (scenario 1), the flow exceeding the threshold is immediately penalized and controlled to avoid overwhelming the bandwidth. The DDoS attack traffic is penalized each time it violated the threshold. Then, the attack was mitigated at the third threshold violation since Max_TVC was reached. Legitimate traffic from hosts 1 and 2 was slightly affected when the attack launched. However, the attack's effects on the legitimate traffic were alleviated when the control algorithm started the penalization. In contrast, the flow traffic in figure 12 (b) exceeded the threshold without any control (scenario 2), and the maximum bandwidth was reached for most of the transmission time. The DDoS traffic drastically overwhelmed the bandwidth, which significantly affected the legitimate traffic, as shown in the figure.

Furthermore, the proposed algorithm reduced the packet drop ratio because it only dropped some packets by limiting the bandwidth for a specific flow. In other words, instead of dropping the entire flow, the algorithm limited the bandwidth since this flow might be legitimate bursty traffic that should not be dropped. With a lack of flow-level bandwidth control, the network would suffer and fail due to DoS/DDoS attacks as well as some of the normal bursty traffic.

3) BANDWIDTH PROFILE-BASED THRESHOLD

Figure 13 (a) presents the single profile-based threshold (scenario 1), while (b) illustrates the multi profile-based threshold (scenario 2). It can be observed that the traffic flow exceeded the threshold multiple times, which results in dropping the packet due to the proposed bandwidth control mechanism. The single profile-based threshold was calculated on the whole features set (time and byte rate) using equation (4). Consequently, the threshold adaptivity that considers the traffic rate variation over time was not achieved.

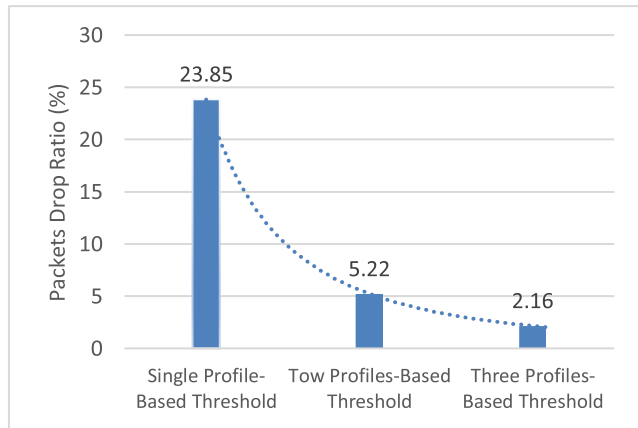


FIGURE 14. Packet drop ratio of bandwidth profile-based threshold.

On the other hand, the multi profile-based threshold shown in figure 13 (b) dynamically switched from moderate to the heavy profile-based threshold when the heavy traffic time started. As a result, the threshold's adaptivity was achieved, the variation of traffic rate over time was considered, and the packets drop ratio was enhanced. Figure 14 displays the packet drop ratio when applying a single profile-based threshold compared to a multi profile-based threshold. The packet drop ratio of using the single profile-based threshold was 23.85%, while it dropped to 2.16% when applying the multi profile-based threshold. Consequently, the packet loss rate was drastically decreased as the number of bandwidth profiles increases.

VII. DISCUSSION

DDoS attacks pose a severe threat to SDN. On the one hand, such an attack is hard to be distinguished since it is usually performed alongside legitimate traffic. Therefore, there is a desperate need for a robust and well-trained learning model (XGBoost) that can accurately classify the traffic with low error. On the other hand, DDoS attacks render the network resources unavailable by flooding and overloading the network with massive traffic that the network resources cannot accommodate. Hence, it is significant to have flow-level bandwidth control alongside the classification model to mitigate the effects of the DDoS attacks on the network resources.

The classification model should be efficiently trained with well-preprocessed datasets that contain a realistic normal and attack traffic. Consequently, we have used three datasets to train the learning models. These datasets were preprocessed using the steps mentioned in section V, namely, data cleaning, transformation, features selection, and class distribution balancing. The results, figures 7-9, show that the learning models trained with the preprocessed dataset attained a superior performance compared to those reported in [33]. Therefore, the preprocessing stage used in this paper has a significant influence on the algorithms' performance.

The evaluation of the binary and multiclass classification, shown in tables 6-13, indicates that the XGBoost classifier

achieved superior scores in the performance metrics of accuracy, recall, precision, and F1-Measure compared to the scores obtained by the RF, DT, NB, and LR classifiers. Additionally, the XGBoost model obtained outstanding results compared with other algorithms in terms of the FPR, with a value close to zero, as displayed in figure 11. The algorithm operational procedures, the ability to select the best parameters, and the correlation of the features used from the dataset have a considerable influence on all models' performance. Therefore, the XGBoost classifier overcame the overfitting problem and attained full utilization of the computational resources. This algorithm performed parallel tree boosting, which quickly and accurately addressed various data science problems. Thus, the results indicate that employing XGBoost in our proposed system yielded high performance in distinguishing normal traffic and DDoS attacks with high accuracy and low error in the SDN context.

The bandwidth (packets/s) is an essential component in a network targeted by DDoS attacks. Therefore, controlling the bandwidth improves the robustness of the network against DDoS attacks by controlling traffic flow to SDN network resources. Consequently, the flow-level bandwidth control proposed in this study using the proposed bandwidth control mechanism significantly limited the effect of DDoS attacks on the network and aided attack detection in SDN. The results presented in section VI, figures 12-13, indicate that the techniques used in this study, namely, the adaptive threshold technique and bandwidth control algorithm, controlled the traffic and preserved the bandwidth even when the network was under attack. The adaptivity of the bandwidth threshold considered both attack traffic and normal bursty traffic. In addition, the bandwidth control algorithm protected the bandwidth from depletion, as demonstrated by the experimental results.

The bandwidth multi profile-based threshold proposed in this paper showed good performance in reducing the packet drop ratio. Figure 14 presents the effectiveness of a multi profile-based threshold over a single profile one (static threshold). The legitimate packet drop ratio dramatically enhanced as the number of the bandwidth profiles increases. Further, multiple bandwidth profiles ensure the threshold adaptivity that noticeably considers the network traffic variation over time.

Additionally, the proposed system performance was validated in a real SDN environment. The XGBoost algorithm achieved superior performance with very high accuracy, precision, recall, F1-measure, and error rate close to zero, as presented in figure 11. Moreover, the bandwidth control algorithm overcame the bandwidth depletion caused by DDoS traffic and improved the bandwidth utilization, as depicted in figure 12 (a) in the results.

In summary, the collaboration between the bandwidth control mechanism and XGBoost model provided a solution that can detect and mitigate DDoS attacks in SDN with high detection efficiency, low error rate, and enhanced resource utilization. In other words, relying on a highly accurate

classification model alone is insufficient to defend the network from advanced attack tools.

VIII. CONCLUSION

In this paper, we propose a defense approach against DDoS attacks for SDN based on the bandwidth control mechanism and XGBoost algorithm. This solution utilizes the bandwidth control mechanism, which employs an adaptive threshold technique and a bandwidth control algorithm. The adaptive threshold is based on multiple bandwidth profiles. The use of multiple bandwidth profiles in setting the threshold ensures the threshold's adaptivity and accuracy to consider the network traffic variation and reduce the packets drop ratio. The packet loss ratio is drastically diminished as the number of the bandwidth profile increases. As shown in the experiment conducted in this paper, the packet loss ratio of a single profile-based threshold was reduced from 23.85% to 2.16% with a multi profile-based threshold.

Further, the bandwidth control algorithm is used together with the adaptive threshold to control traffic flow to the network, which can significantly reduce the impact of DDoS traffic on the network resources, such as bandwidth depletion. This control algorithm uses a penalty mechanism to limit the bandwidth for a specific flow violating the threshold. Consequently, this control algorithm enhances the utilization of the network resources, as discussed earlier.

Our proposed solution then employs a trigger-based classification and detection model based on the XGBoost algorithm to distinguish different types of DDoS attacks from the normal traffic. The XGBoost model is evaluated using the CICDDoS2019, NSL-KDD, and CAIDA datasets. These datasets are subjected to an efficient preprocessing stage to prevent misleading accuracy and overfitting problems. Then, the evaluation of the binary and multiclass classification of the XGBoost is conducted.

Furthermore, the proposed system is validated in real-time with the SDN environment. The validation demonstrates that the proposed system attains good performance with 99.9% accuracy, a low FPR and packet loss ratio, and improved resource utilization in the SDN context. Moreover, employing the XGBoost algorithm enhances DDoS attack detection and mitigation with high accuracy, precision, recall, F1 scores, and the lowest FPR, among other algorithms, such as RF and DT. The bandwidth, which is the main target of any DDoS attack, is preserved and fully utilized owing to the proposed bandwidth control mechanism. In future work, the TVC will be investigated to attain optimal settings to achieve high detection accuracy and low processing and communication overhead. We also intend to deploy our approach in a multi-controller based SDN environment.

CONFLICTS OF INTEREST

The authors declare that there is no conflict of interest regarding the publication of this paper.

ACKNOWLEDGMENT

The authors would like to express their gratitude to King Abdulaziz University, KSA, for providing the necessary facilities for conducting the research.

REFERENCES

- [1] K. Kalkan, G. Gur, and F. Alagoz, "Defense mechanisms against DDoS attacks in SDN environment," *IEEE Commun. Mag.*, vol. 55, no. 9, pp. 175–179, 2017.
- [2] S. Dong, K. Abbas, and R. Jain, "A survey on distributed denial of service (DDoS) attacks in SDN and cloud computing environments," *IEEE Access*, vol. 7, pp. 80813–80828, 2019.
- [3] S. Singh and S. Prakash, "A survey on software defined network based on architecture, issues and challenges," *Proc. 3rd Int. Conf. Comput. Methodol. Commun. (ICCMC)*, 2019, pp. 568–573.
- [4] K. S. Sahoo, B. K. Tripathy, K. Naik, S. Ramasubbareddy, B. Balusamy, M. Khari, and D. Burgos, "An evolutionary SVM model for DDOS attack detection in software defined networks," *IEEE Access*, vol. 8, pp. 132502–132513, 2020.
- [5] C.-H. Lin, C.-Y. Li, and K. Wang, "Setting malicious flow entries against SDN operations: Attacks and countermeasures," in *Proc. IEEE Conf. Dependable Secure Comput. (DSC)*, Dec. 2018, pp. 1–8.
- [6] A. L. Aliyu, P. Bull, and A. Abdallah, "Performance implication and analysis of the OpenFlow SDN protocol," in *Proc. 31st Int. Conf. Adv. Inf. Netw. Appl. Workshops*, Mar. 2017, pp. 391–396.
- [7] K. S. Sahoo, D. Puthal, M. Tiwary, J. J. P. C. Rodrigues, B. Sahoo, and R. Dash, "An early detection of low rate DDoS attack to SDN based data center networks using information distance metrics," *Future Gener. Comput. Syst.*, vol. 89, pp. 685–697, Dec. 2018.
- [8] K. Benzekki, A. El Fergougui, and A. E. Elaloui, "Software-defined networking (SDN): A survey," *Secur. Commun. Netw.*, vol. 9, no. 18, pp. 5803–5833, Dec. 2016.
- [9] K. Kalkan, L. Altay, G. Gur, and F. Alagöz, "JESS: Joint entropy-based DDoS defense scheme in SDN," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 10, pp. 2358–2372, Oct. 2018.
- [10] K. B. Virupakshar, M. Asundi, K. Channal, P. Shettar, S. Patil, and D. G. Narayan, "Distributed denial of service (DDoS) attacks detection system for OpenStack-based private cloud," *Procedia Comput. Sci.*, vol. 167, pp. 2297–2307, 2020.
- [11] Q. Yan, F. R. Yu, Q. Gong, and J. Li, "Software-defined networking (SDN) and distributed denial of service (DDoS) attacks in cloud computing environments: A survey, some research issues, and challenges," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 1, pp. 602–622, 1st Quart., 2016.
- [12] Z. Wang, H. Hu, and C. Zhang, "On achieving SDN controller diversity for improved network security using coloring algorithm," in *Proc. 3rd IEEE Int. Conf. Comput. Commun. (ICCC)*, Dec. 2017, pp. 1270–1275.
- [13] C. Gkountis, M. Taha, J. Lloret, and G. Kambourakis, "Lightweight algorithm for protecting SDN controller against DDoS attacks," in *Proc. IEEE Wireless Mobile Netw. Conf.*, Sep. 2017, pp. 1–6.
- [14] M. Idhammad, K. Afdel, and M. Belouch, "Detection system of HTTP DDoS attacks in a cloud environment based on information theoretic entropy and random forest," *Secur. Commun. Netw.*, vol. 2018, Jun. 2018, Art. no. 1263123.
- [15] J. Cheng, C. Zhang, X. Tang, V. S. Sheng, Z. Dong, and J. Li, "Adaptive DDoS attack detection method based on multiple-kernel learning," *Secur. Commun. Netw.*, vol. 2018, pp. 1–19, Oct. 2018.
- [16] A. Raj, A. S. Bhat, and L. V. Nambhothiri, "Effective threshold defence against DOS attack on SDN controller," *Int. J. Pure Appl. Math.*, vol. 119, no. 10, pp. 691–698, 2018.
- [17] J. David and C. Thomas, "Efficient DDoS flood attack detection using dynamic thresholding on flow-based network traffic," *Comput. Secur.*, vol. 82, pp. 284–295, May 2019.
- [18] B. Jia, X. Huang, R. Liu, and Y. Ma, "A DDoS attack detection method based on hybrid heterogeneous multiclassifier ensemble learning," *J. Electr. Comput. Eng.*, vol. 2017, pp. 1–9, Jan. 2017.
- [19] T. Wang and H. Chen, "SGuard: A lightweight SDN safe-guard architecture for DoS attacks," *China Commun.*, vol. 14, no. 6, pp. 113–125, 2017.
- [20] J. Ye, X. Cheng, J. Zhu, L. Feng, and L. Song, "A DDoS attack detection method based on SVM in software defined network," *Secur. Commun. Netw.*, vol. 2018, pp. 1–8, Apr. 2018.

- [21] B. Han and X. Yang, "OverWatch: A cross-plane DDoS attack defense framework with collaborative intelligence in SDN," *Secur. Commun. Netw.*, vol. 2018, p. 15, Jan. 2018.
- [22] B. V. Karan, D. G. Narayan, and P. S. Hiremath, "Detection of DDoS attacks in software defined networks," in *Proc. 3rd Int. Conf. Comput. Syst. Inf. Technol. Sustain. Solution (CSITSS)*, 2018, pp. 265–270.
- [23] R. Santos, D. Souza, W. Santo, A. Ribeiro, and E. Moreno, "Machine learning algorithms to detect DDoS attacks in SDN," *Concurrency Comput. Pract. Exper.*, vol. 32, no. 16, pp. 1–14, Aug. 2020.
- [24] M. M. Oo, S. Kamolphiwong, T. Kamolphiwong, and S. Vasupongayya, "Advanced support vector machine (ASVM)-based detection for distributed denial of service (DDoS) attack on software defined networking (SDN)," *Comput. Netw. Commun.*, vol. 2019, p. 12, Mar. 2019.
- [25] T. V. Phan and M. Park, "Efficient distributed denial-of-service attack defense in SDN-based cloud," *IEEE Access*, vol. 7, pp. 18701–18714, 2019.
- [26] Y. Wang, T. Hu, G. Tang, J. Xie, and J. Lu, "SGS: Safe-guard scheme for protecting control plane against DDoS attacks in software-defined networking," *IEEE Access*, vol. 7, pp. 34699–34710, 2019.
- [27] K. S. Arpitha, "DDoS attacks using machine learning," *J. Xi'an Univ. Archit. Technol.*, vol. 2, no. 4, pp. 3380–3384, 2020.
- [28] J. A. Pérez-Díaz, I. A. Valdovinos, K.-K. R. Choo, and D. Zhu, "A flexible SDN-based architecture for identifying and mitigating low-rate DDoS attacks using machine learning," *IEEE Access*, vol. 8, pp. 155859–155872, 2020.
- [29] V. Deepa, K. M. Sudar, and P. Deepalakshmi, "Detection of DDoS attack on SDN control plane using hybrid machine learning techniques," in *Proc. Int. Conf. Smart Syst. Inventive Technol. (ICSSIT)*, 2019, pp. 299–303.
- [30] M. T. Manavi, "Defense mechanisms against distributed denial of service attacks: A survey," *Comput. Electr. Eng.*, vol. 72, pp. 26–38, Nov. 2018.
- [31] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, vols. 13–17, pp. 785–794, 2016.
- [32] A. Natekin and A. Knoll, "Gradient boosting machines, a tutorial," *Frontiers Neuroinformatics*, vol. 7, p. 21, Dec. 2013.
- [33] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *Proc. Int. Carnahan Conf. Secur. Technol.*, Oct. 2019, pp. 1–8.
- [34] L. Dhanabal and S. P. Shantharajah, "A study on NSL-KDD dataset for intrusion detection system based on classification algorithms," *Int. J. Adv. Res. Comput. Commun. Eng.*, vol. 4, no. 6, pp. 446–452, 2015.
- [35] The CAIDA UCSD. *DDoS Attack 2007*. Accessed: Sep. 9, 2020. [Online]. Available: https://www.caida.org/data/passive/ddos-20070804_dataset.xml
- [36] I. Sharafaldin, A. Gharib, A. H. Lashkari, and A. A. Ghorbani, "Towards a reliable intrusion detection benchmark dataset," *Softw. Netw.*, vol. 2017, no. 1, pp. 177–200, 2017.
- [37] D. Lashkari. (2019). *GitHub*. [Online]. Available: <https://github.com/ahlashkari/CICFlowMeter>
- [38] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of TOR traffic using time based features," in *Proc. 3rd Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Sep. 2017, pp. 253–262.
- [39] M. Tavallae, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl. (CISDA)*, Jul. 2009, pp. 1–6.
- [40] S. Das, D. Venugopal, S. Shiva, and F. T. Sheldon, "Empirical evaluation of the ensemble framework for feature selection in DDoS attack," in *Proc. 7th IEEE Int. Conf. Cyber Secur. Cloud Comput. (CSCloud)*, New York, NY, USA, 2020, pp. 56–61.
- [41] D. Gnanambal, D. Thangaraj, V. T. Meenatchi, and D. Gayathri, "Classification algorithms with attribute selection: An evaluation study using WEKA," *Int. J. Adv. Netw. Appl.*, vol. 9, no. 6, pp. 363–3644, 2018.
- [42] A. More, "Survey of resampling techniques for improving classification performance in unbalanced datasets," *Mach. Learn.*, vol. 10000, pp. 1–7, Aug. 2016.
- [43] B. Kovács, F. Tinya, C. Németh, and P. Ódor, "Unfolding the effects of different forestry treatments on microclimate in oak forests: Results of a 4-yr experiment," *Ecol. Appl.*, vol. 30, no. 2, pp. 321–357, 2020.
- [44] I. Tomek, "Tomek link: Two modifications of CNN," *IEEE Trans. Syst. Man Cybern.*, vol. SMC-6, no. 11, pp. 769–772, Nov. 1976.
- [45] A. Fernández, S. García, M. J. del Jesus, and F. Herrera, "A study of the behaviour of linguistic fuzzy rule based classification systems in the framework of imbalanced data-sets," *Fuzzy Sets Syst.*, vol. 159, no. 18, pp. 2378–2398, Sep. 2008.
- [46] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, and B. Thirion, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Jan. 2011.
- [47] S. Bhatia, G. Mohay, A. Tickle, and E. Ahmed, "Parametric differences between a real-world distributed Denial-of-Service attack and a flash event," in *Proc. 6th Int. Conf. Availability, Rel. Secur. (ARES)*, Aug. 2011, pp. 210–217.
- [48] S. Bhatia, D. Schmidt, G. Mohay, and A. Tickle, "A framework for generating realistic traffic for distributed Denial-of-Service attacks and flash events," *Comput. Secur.*, vol. 40, pp. 95–107, Feb. 2014.
- [49] H. Zhang, "Exploring conditions for the optimality of Naïve Bayes," *Int. J. Pattern Recognit. Artif. Intell.*, vol. 19, no. 2, pp. 183–198, 2005.
- [50] N. Statistical and S. Ncss, "Logistic regression: From introductory to advanced concepts and applications," *Choice Rev. Online*, vol. 47, no. 1, pp. 47–338–47–338, 2009.
- [51] H. H. Patel and P. Prajapati, "Study and analysis of decision tree based classification algorithms," *Int. J. Comput. Sci. Eng.*, vol. 6, no. 10, pp. 74–78, Oct. 2018.
- [52] M. Denil, D. Matheson, and N. De Freitas, "Narrowing the gap: Random forests in theory and in practice," *Proc. 31st Int. Conf. Mach. Learn. (ICML)*, 2014, vol. 2, no. 1998, pp. 1006–1016.
- [53] Ryu. Accessed: Jun. 29, 2020. [Online]. Available: <https://ryu.readthedocs.io/en/latest/index.html>
- [54] F. Ketikci and S. Askar, "Emulation of software defined networks using mininet in different simulation environments," in *Proc. 6th Int. Conf. Int. Syst., Modeling Simulation*, Kuala Lumpur, Malaysia, Feb. 2015, pp. 205–210.
- [55] P. Kaviani and S. Dhotre, "International journal of advance engineering and research short survey on Naïve Bayes algorithm," *Int. J. Adv. Eng. Res. Dev.*, vol. 4, no. 11, pp. 607–611, 2017.
- [56] D. Berrar, "Bayes' theorem and Naïve Bayes classifier," *Encycl. Bioinforma. Comput. Biol. ABC Bioinforma.*, vols. 1–3, pp. 403–412, Sep. 2018.
- [57] M. Maalouf, "Logistic regression in data analysis: An overview," *Int. J. Data Anal. Tech. Strategies*, vol. 3, no. 3, pp. 281–299, 2011.



HASSAN A. ALAMRI received the B.Sc. degree in computer science from King Abdulaziz University, Jeddah, Saudi Arabia, in 2010, and the M.S. degree in internetworking from Dalhousie University, Halifax, NS, Canada, in 2013. He is currently pursuing the Ph.D. degree in computer science with King Abdulaziz University. His research interests include software-defined networking and network security.



VIJAY THAYANANTHAN received the Ph.D. degree in engineering (digital communication engineering) from the Department of Communication Systems, University of Lancaster, U.K., in 1998. Since 2000, he has been working as a Research Engineer and a Senior Algorithm Development Engineer with Advantech Ltd., Southampton University Science Park, U.K., and Amfax Ltd., U.K. He is currently a Professor with the Computer Science Department, King Abdulaziz University, Jeddah, Saudi Arabia. His research interests include wireless communication algorithm design and mobile communication analysis, security management of communication network and big data, computer security, and wireless sensor networks. Since 2005, he has been a full-time Member of the Institution of Engineering Technology (IET), U.K.

...